

赋能工程师与 Codex App: OpenAI 谈 AI 工程的未来

Alexander Emiricos & Roman Huitt (OpenAI) ——Empowering Engineers & The Codex App

Alexander Emiricos & Roman Huitt (OpenAI)

2026 年 7 月 10 日



视频作者/频道: AI Engineer

发布日期: 2026-07-01

视频时长: 原视频 8:36:51; 本笔记对应 00:28:37-00:48:09 (约 19 分 32 秒)

视频链接: WF2026:SoftwareFactories&Keynotes 00:28:37

目录

1 开场：工程师没有消失，AI 工程师正在重塑世界	2
1.1 从世界博览会到 AI Engineer World's Fair：把未来公开地建造出来	2
1.2 “AI 工程师正在接管世界”——为什么工程师反而更重要	3
1.3 模型迭代速度：从每 15 个月到每 6 周	4
1.4 本章小结	4
2 从自动补全到自主执行：AI 工程范式的跃迁	5
2.1 罗马的两次 DevDay 现场演示：2024 无人机界面 vs 2025 摄像/灯光系统	5
2.2 能力阶梯：completion → inline prediction → Command K → build/test loop → long-horizon goals	7
2.3 2026 年已经交付的关键形态：Codex App、Goal Mode、Remote	8
2.4 本章小结	8
3 Codex App 产品形态：让聊天与深度协作并存	9
3.1 产品哲学：不是自动化工程师，而是最大化赋能工程师	9
3.2 两种交互模式：Chat 与 Hands-on	10
3.3 Codex App 示例：从世界杯比分预测到具体修改	12
3.4 为什么需要一个新的协作界面，而不是 CLI 或 IDE	13
3.5 本章小结	14
4 OpenAI 的开放分层栈：每个人都可以 fork 同一套东西	15
4.1 核心原则：每一层都使用我们交给开发者的同一套接口	15
4.2 Codex Harness 开源与 agents.md	16
4.3 App Server 与第三方客户端（Open Code、Codex Monitor、VS Code 扩展）	18
4.4 插件扩展点：browser use、computer use、特定领域插件	18
4.5 本章小结	20
5 让智能落地：成本、速度、运行环境与价值最大化	21
5.1 Value maxing：从 token maxing 到价值最大化	21
5.2 成本效率：TerminalBench 2.1 与 GPT-5.6 系列模型	21
5.3 速度：GPT-5.6 Sol on Cerebras 750 tokens/sec	22
5.4 运行环境：本地、云端、以及未来的无感选择	24
5.5 Theo 的预言与 OpenAI 的回应	24
5.6 本章小结	25
6 总结与延伸	25
6.1 演讲者核心主张回顾	26
6.2 我们的提炼：工程师、智能体、开放栈、价值最大化	27
6.3 开放问题与下一步	27
6.4 本章小结	28

1 开场：工程师没有消失，AI 工程师正在重塑世界

本节核心主张

软件吞噬世界之后，AI 正在吞噬软件；但工程师并未消失，而是正在以“AI 工程师”的身份重新成为推动前沿（frontier）的力量。本节用世界博览会（World’s Fair）的类比、两次现场演示的对比，以及模型迭代速度的数据，建立全篇的基调：未来不是被某种神秘力量送达的，而是被这间屋子里的工程师公开地建造出来的。

1.1 从世界博览会到 AI Engineer World’s Fair：把未来公开地建造出来

Roman Huitt 与 Alexander Emiricos 在 00:28:37 登台。Roman 首先指出，当天到场的不只是演讲者，更是“超过 7000 名 AI 工程师”——真正每天都在使用并推动技术边界的人。演讲者将这场活动比作历史上的 World’s Fair（世界博览会）：那些原本听起来不可能的想法，被公开地建造出来，让人们看得见、走得进，最终也相信它真的会发生。

World’s Fair 的教学意义

世界博览会并不是单纯的展览，而是把“未来”做成可被公众访问的实体。Roman 想用这个类比说明：AI 工程的未来不是从某个遥远的地方“到达”的，而是由在场的工程师在这里、现在、公开地建造出来的。这种“building in public”的姿态，也贯穿了 OpenAI 后面谈到的开放分层栈。



图 1：活动现场主舞台：AI Engineer World’s Fair 的开幕氛围与赞助商标识。

从 00:29:55 到 00:30:05，Roman 明确说道：

⁰视频画面时间区间：00:28:37–00:29:10。

⁰视频画面时间区间：00:29:30–00:30:05。



图 2: 世界博览会历史照片: 演讲者用“公开建造未来”的类比, 说明 AI 工程的未来正在这里被实时创造。

原始对话片段 (00:29:55–00:30:05)

Roman Huitt: The future of engineering is not arriving from somewhere else. It's really being built here by the people in this room and much faster than most expected.

这句话是整场演讲的基调。它不是一句口号, 而是后面所有产品决策 (Codex App、开放栈、价值最大化) 的前提: 如果未来是公开建造的, 那么工具和基础设施就应该让尽可能多的人参与建造, 而不是把能力封闭在少数人手里。

1.2 “AI 工程师正在接管世界”——为什么工程师反而更重要

不要误解的常见论调

当时流行一种说法: 代码正在被抽象掉, 所以工程师终将消失。Roman 在 00:30:06 直接回应了这个论调, 并称事实“恰恰相反”。本节要澄清的是: AI 不是在替代工程师, 而是在扩大工程师的杠杆。

Roman 在 00:30:20 用一句话概括了历史弧线: “software ate the world and then AI ate software. But now what we're here to say is that the AI engineers are eating the world.” 这句话的层次很清楚:

- 软件吞噬世界: Marc Andreessen 的经典论断, 指软件进入各行各业;
- AI 吞噬软件: AI 开始成为软件本身的核心能力;
- AI 工程师吞噬世界: 真正塑造这一轮变革的, 是懂得使用 AI 的工程师。

Alexander 在 00:30:45 进一步解释: 工程从来不是“写代码”, 而是“解决问题”——把最新科学与设计、品味、判断力和想象力结合起来, 做出人们真正可用的东西。因此, 这不是工程的终结, 而是“回归工程的根源”。

AI 工程师的定义（本节首次出现）

在本演讲的语境中，**AI 工程师**不只是会调用模型 API 的程序员，而是能够把智能体（agent）接入真实 workflow、持续推动产品边界的人。他们的核心任务不是替代自己，而是让工具承担更多可自动化的部分，从而把精力放回判断、设计与创造。

演讲中，Roman 用 “AI engineers are the people here pushing the frontier” 来描述这群人。这里的 frontier（前沿）不是抽象概念，而是指 “当前最先进、最能创造新能力的模型与实践”。随后 Alexander 将 “他们如何把这种新能力带给每个人” 作为工程师的使命，这也为第 3 节讨论 Codex App 的产品形态埋下了伏笔。

1.3 模型迭代速度：从每 15 个月到每 6 周

技术的加速度是支撑上述论断的硬证据。Roman 在 00:31:16 给出了一个具体数字：OpenAI 过去大约每 15 个月发布一次新模型，现在大约是每 6 周。同时他还提到，就在演讲前一周，5.6 系列模型已经发布预览版。

模型迭代速度的数据点

- 过去：约每 15 个月发布一个新模型；
- 现在：约每 6 周发布一次；
- 演讲当周：GPT-5.6 系列预览版已经上线。

来源：00:31:16–00:31:32。

Alexander 紧接着在 00:31:32 指出，模型之上的产品进步同样 “relentless”（relentless 即 “不停歇的”）。模型更快、工具更强，工程的感受已经完全不同。这一速度变化不是孤立的技术新闻，而是后面 “能力阶梯” 论述的底层驱动力：正因为模型每 6 周就上一个台阶，过去两年才能从 “自动补全” 一路走到 “长程目标自主执行”。

从速度到工程范式的跃迁

模型迭代速度加快意味着：

1. 产品团队可以更快地把模型能力转化为新功能；
2. 工程师的判断力变得比 “写代码速度” 更稀缺；
3. 工具必须围绕 “持续升级” 设计，而不是假设某一版模型是终点。

1.4 本章小结

本节为整篇笔记奠定了三个基本观点：

1. **未来是公开建造的**。World’s Fair 的类比说明，AI 工程的未来不是从外部降临，而是由工程师在活动现场、在开源社区、在产品迭代中实时创造出来的。

2. **工程师没有消失，反而更重要。**软件吃掉了世界，AI 吃掉了软件，但最终“AI 工程师”正在吃掉世界。因为工程的本质是解决问题，而不是写代码。
3. **模型速度已经把工程范式带上新轨道。**从每 15 个月到每 6 周，模型迭代的速度让“从自动补全到自主执行”的跃迁成为可能，也让工具设计必须面向持续升级而非一次性交付。

这三个观点会在第 2 节的能力阶梯、第 3 节的 Codex App、第 4 节的开放分层栈以及第 5 节的价值最大化中反复回响。

2 从自动补全到自主执行：AI 工程范式的跃迁

本节核心主张

过去两三年，AI 辅助编程经历了从“补全一行代码”到“自主完成长程目标”的跃迁。这个变化不是简单的功能叠加，而是模型开始具备“验证自己工作”的能力，从而形成了 build/test loop（构建-测试循环）。本节用 Roman 两次 DevDay 现场演示的对比，展示这一跃迁的实质，并列出了 2026 年已经交付的关键形态：Codex App、Goal Mode、Remote。

2.1 罗马的两次 DevDay 现场演示：2024 无人机界面 vs 2025 摄像/灯光系统

Alexander 在 00:31:41 开始回顾自己“连续几段让人感到 mind-blowing 的经历”。他说，很长一段时间里，大家只有 completion（自动补全），然后是 inline prediction（行内预测），再然后是 Command K（按 K 唤出命令改代码），但这些阶段的模型“不会测试自己的工作”。直到模型开始测试自己的工作，再到现在模型能够承担“长程且困难的目标”。

Roman 在 00:32:10 接过这个话题，并用自己的两次 DevDay 现场演示来说明变化。他说：

原始对话片段（00:32:10–00:32:56）

Roman Huitt: I can't believe that build and test loop was not even part of the models just two years ago. This was a picture of me at DevDay 2024 and I used o1 in preview at the time to build a mini drone interface from scratch. And the slightly insane part is the model could not actually run the code or verify its own work. ... By DevDay of last year in 2025, I was confident enough now the models could test their own work to kind of control an entire camera system and lighting system live.

这两次演示的关键差异在于：

- **2024 年 DevDay:** 模型能生成代码，但不能运行或验证代码。Roman 需要“cross my fingers”（祈祷）才能保证现场演示成功。
- **2025 年 DevDay:** 模型已经能够测试自己的工作，因此 Roman 敢让模型现场控制一整套摄像和灯光系统。

这两张截图的对比，是“AI 工程范式跃迁”最直观的视觉证据。2024 年的模型是“写完了交给你”，2025 年的模型是“写完了自己跑、自己测、自己调”。这种变化正是“智能体化的”（agentic）能力的开端：模型不再只是响应单次请求，而是能够在多轮迭代中持续趋近目标。

⁰ 视频画面时间区间：00:32:15–00:32:50。

⁰ 视频画面时间区间：00:32:50–00:33:05。



图 3: 2024 年 DevDay 现场演示: Roman 用 o1 预览版从零编写无人机界面, 但模型当时无法运行或验证自己生成的代码。

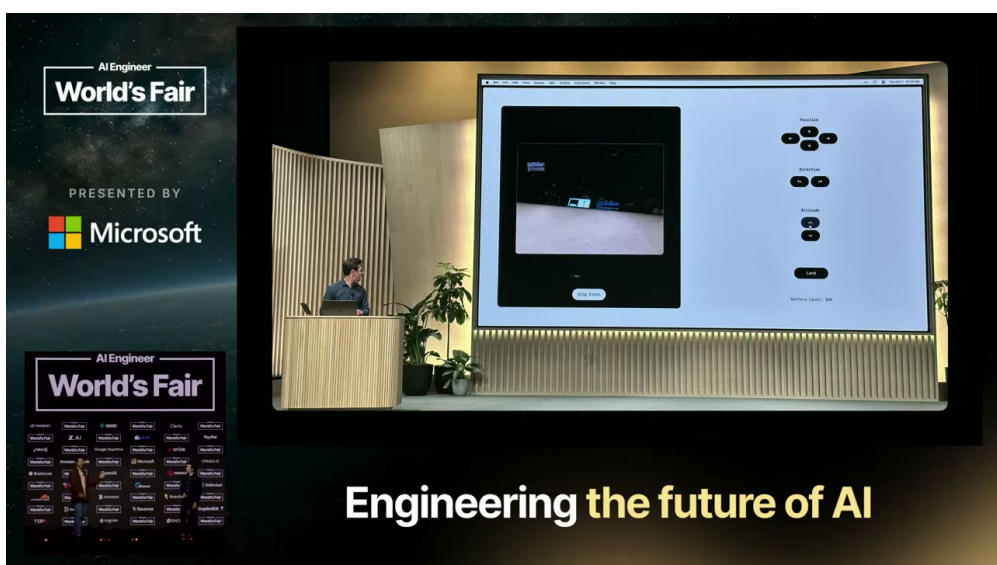


图 4: 2025 年 DevDay 现场演示: 模型已能测试自己的工作, Roman 让它现场控制摄像与灯光系统。

不要把“演示成功”当成魔术

Roman 自己提到，2024 年的 demo 他只能估计“大部分时间能工作，但不是所有时候”。2025 年他能够自信，不是因为模型变得“绝对正确”，而是因为它能够检测并修正自己的错误。这是工程可靠性的关键转折：从“人兜底”变成“模型自己闭环”。

2.2 能力阶梯：completion → inline prediction → Command K → build/test loop → long-horizon goals

Alexander 在 00:31:47 到 00:32:10 用一个连续的能力阶梯概括了这段历史。我们可以把它整理成下面的阶段表：

表 1: AI 辅助编程能力阶梯（来源：00:31:47–00:32:10）

阶段	中文说明	关键特征
completion	自动补全	模型预测下一行或下一段代码
inline prediction	行内预测（inline prediction）	在光标处直接生成代码，更无缝
Command K	按 K 唤出命令改代码	用户发出指令，模型修改代码
build/test loop	构建-测试循环（build/test loop）	模型能运行测试并验证自己的工作
long-horizon goals	长程目标	模型持续工作，直到完成复杂目标

什么是“智能体化的”能力？

“智能体化的”（agentic）不是一个营销词，而是一个行为描述：模型不再只是对单次输入给出单次输出，而是能够在多个步骤中感知环境（例如编译结果、测试日志、浏览器状态）、调用工具、修正方向，并持续运行直到达成目标。build/test loop 是这个能力的基础，因为模型需要能判断“我做得对不对”。

这个阶梯的每一步都建立在“模型能否验证自己工作”这一关键分水岭上。前面的 completion、inline prediction、Command K 阶段，模型本质上都是“生成器”：它把代码写给你，但不知道代码是否能跑。进入 build/test loop 阶段后，模型变成了“闭环系统”：它生成、运行、读取反馈、再生成。这个闭环正是“长程目标”的前提——如果一个模型不能判断自己走到哪一步，它就不可能持续向一个 distant goal 推进。

能力阶梯的教学要点

- 1. 前三个阶段（completion、inline prediction、Command K）解决的是“写代码更快”的问题；
- 2. build/test loop 解决的是“写对了没有”的问题；
- 3. long-horizon goals 解决的是“能否独立完成复杂任务”的问题。

2.3 2026 年已经交付的关键形态：Codex App、Goal Mode、Remote

Roman 在 00:33:06 感慨 “今年 alone has been crazy”（仅今年一年就足够疯狂）。他随后提到自己最喜欢的三个已交付形态：Codex App、Goal Mode、Remote。这些形态不是孤立的功能，而是上述能力阶梯在 2026 年的产品化落地：

- **Codex App**：一个以聊天为主、可深度介入代码的协作式应用。它将在第 3 节详细展开。
- **Goal Mode**：让模型以完成一个目标为导向、持续自主工作直到目标达成的模式。它对应能力阶梯中的 long-horizon goals。
- **Remote**：让智能体在云端/远程环境中运行任务，而不是占用用户本地机器的工作模式。它将在第 5 节的运行环境讨论中再次出现。

Roman 在 00:33:31 补充了一句关键的话：“obviously we couldn’t do these things if we didn’t use Codex to build Codex.” 这说明 Codex 不只是对外发布的产品，也是 OpenAI 内部用来构建 Codex 自身的工具。这种“自举”现象在软件史中并不罕见，但它意味着：Codex 的能力边界会直接转化为 OpenAI 产品迭代的速度。

Alexander 在 00:33:35 紧接着提出，现在的智能体（agent）已经能够完成“你能在自己电脑上做的任何任务”。这意味着它们不只帮你写代码，还帮你处理“编码之前”和“编码之后”的工作。他特别提到：

原始对话片段（00:33:50–00:34:08）

Alexander Emiricos: If you can connect the agent to not only the work that you have to do, but why it has to be done, that’s how you can get the agent to start to begin much more work. And then if you can connect it to what you do afterwards, review and deploy, that’s how you help it land much more work.

这段话为下一节（第 3 节）的 Codex App 产品形态埋下了核心线索：一个好的智能体协作界面，必须把“为什么做”（规划）、“做什么”（执行）、“做得对不对”（review/deploy）串成一个完整的闭环。这也呼应了第 1 节中“AI 工程师没有消失”的论断：工程师的价值正体现在把智能体连接到正确的上下文与判断上。

2026 年的关键形态不是终点

Codex App、Goal Mode、Remote 是 2026 年已经交付的三种形态，但它们代表的不是最终答案，而是同一套能力阶梯的三种产品化切口。后续章节会分别说明：Codex App 解决“与智能体协作的界面”问题；Goal Mode 解决“长程目标自主执行”问题；Remote 解决“任务在哪里运行”问题。

2.4 本章小结

本节用一条能力阶梯和两次 DevDay 演示的对比，说明了 AI 工程范式从“自动补全”到“自主执行”的跃迁：

1. **2024 vs 2025**：2024 年的模型能写代码但不能验证；2025 年的模型能测试自己的工作，从而敢控制现场硬件系统。这是从“生成器”到“闭环系统”的转变。

2. **能力阶梯**: completion → inline prediction → Command K → build/test loop → long-horizon goals。build/test loop 是关键分水岭, 它让模型获得了判断对错的能力, 也让 long-horizon goals 成为可能。
3. **2026 已交付形态**: Codex App (协作界面)、Goal Mode (长程目标)、Remote (远程运行) 是能力阶梯落地后的三种产品形态。它们不是终点, 而是下一章讨论产品设计的入口。

接下来, 第 3 节将聚焦 Codex App: 为什么它需要是一个“聊天优先、但可深度介入”的新界面, 而不是传统 CLI 或 IDE 的简单扩展。

3 Codex App 产品形态: 让聊天与深度协作并存

在第 2 节中, 我们看到模型已经从单次补全走向了能够自主运行构建-测试循环(build/test loop)并追求长期目标(long-horizon goals)的**智能体 (agent)**。能力曲线本身并不决定产品形态: 当模型几乎可以完成任何中等复杂度的计算机任务时, 工程师与智能体之间的**协作界面**应该长什么样? 本节围绕 OpenAI 给出的答案——**Codex App**——展开产品哲学、双模式交互设计, 以及它为什么既不是传统 CLI, 也不是 IDE 的复制品。

3.1 产品哲学: 不是自动化工程师, 而是最大化赋能工程师

演讲者在 00:33:35 处指出, 真正令他兴奋的不是模型能写代码, 而是**智能体 (agent)**能够完成用户在自己电脑上可以完成的任何任务。这意味着智能体的作用范围不再局限于编码本身, 而是向编码前后延伸:

- **编码之前**: 帮助理解“为什么要做这件事”, 即把任务目标与上下文对齐;
- **编码之后**: 帮助审查 (review) 和部署 (deploy), 让修改真正落地。

为什么“前后”比“中间”更关键

在第 2 节的能力阶梯中, 模型已经能写代码; 但演讲者强调, 如果能把智能体连接到“为什么要做”以及“做完之后怎么办”, 它才能承担更多有价值的工作。换言之, **loop** 的价值不仅在于循环执行, 而在于把执行嵌入完整的工程流程。

这种连接带来了两个层面的收益:

1. **执行速度更快**: 智能体在闭环中自我验证、自我修正, 减少了工程师在“写—测—改”之间的往返成本;
2. **决策质量更高**: 作为产品经理, 演讲者认为最令人兴奋的是团队能够“更快地原型、更久地陪伴用户”, 从而做出更好的产品决策 (00:34:10–00:34:25)。

进入产品形态讨论之前, 演讲者在 00:34:45 做了一个坦率的判断: 对于典型中等复杂度的计算机任务, 如果给模型和人类同样的时间, 模型往往比人类完成得更好。这引出了产品设计的核心张力: 模型已经比人类更能“做事”, 那么我们应该用它来自动化工程师, 还是用它来提升工程师?

OpenAI 的使命锚点

在 00:35:10 处，演讲者调出 OpenAI 的使命——“AGI that benefits all of humanity”——作为产品形态思考的出发点。他认为当前需要回答两个主要问题：

1. 如何让智能体真正在世界上做事？
2. 如何让智能体为我们所用，即产品应该是什么形态？

对于第二个问题，演讲者的答案非常明确：目标**绝不是**自动化工程师，而是**最大化赋能工程师**（**maximally empowers engineers**）。



图 5: OpenAI 使命：让 AGI 造福全人类（00:35:10–00:35:25）

这一定位是理解 Codex App 的钥匙：它不会替代工程师的判断，而是把工程师从重复执行中解放出来，让他们把更多精力放在目标设定、方向审查和创造性决策上。

3.2 两种交互模式：Chat 与 Hands-on

基于“最大化赋能”这一目标，演讲者在 00:35:58 提出产品形态其实可以简化为两种模式：

1. **Chat（聊天）**：一个可以随时随地求助的单一实体；
2. **Hands-on（动手介入）**：一个足够强大的协作式 UI，用于在需要时检查、引导或亲手塑造结果。

Chat 被低估了

演讲者承认，有人认为 chat 已死，但他明确反驳：“I think chat is underrated.”（00:36:02–00:36:08）。在他看来，聊天仍然是最高频、最自然的协作入口，因为大部分时间里，工程师只想“说一下需求，然后让智能体去执行”。

⁰ 视频画面时间区间：00:35:10–00:35:25。

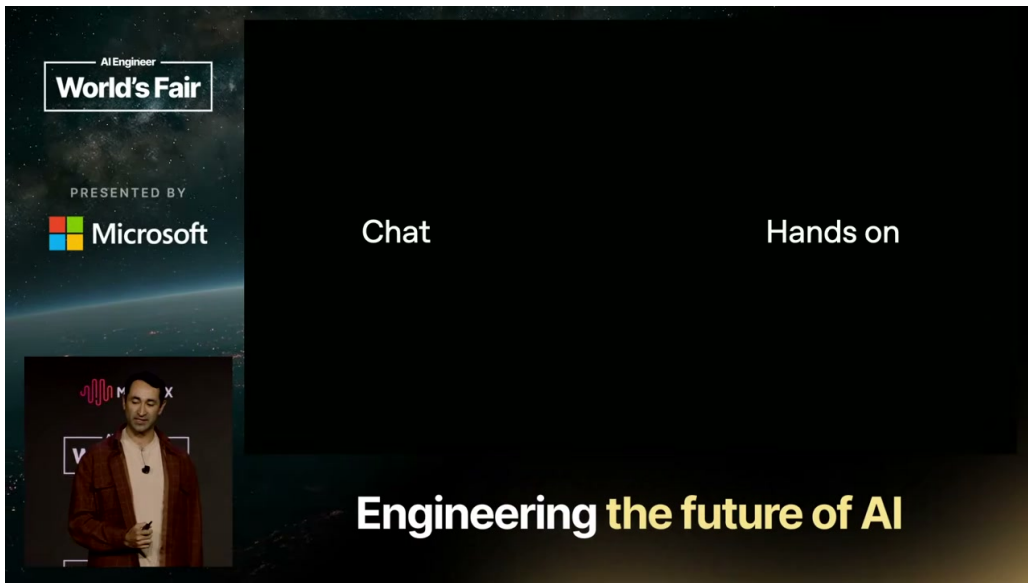


图 6: Codex App 的两种交互模式: Chat 与 Hands-on (00:36:00–00:36:20)

为了让听众理解这两种模式何时切换，演讲者在 00:36:32 使用了“与团队协作”的类比：大部分时候你只是跟团队成员聊天、分配任务，然后让他们去执行；你不需要趴在队友背后盯着每一个工作单元。但每隔一段时间，你又需要一起深入细节，把问题拆解到底。

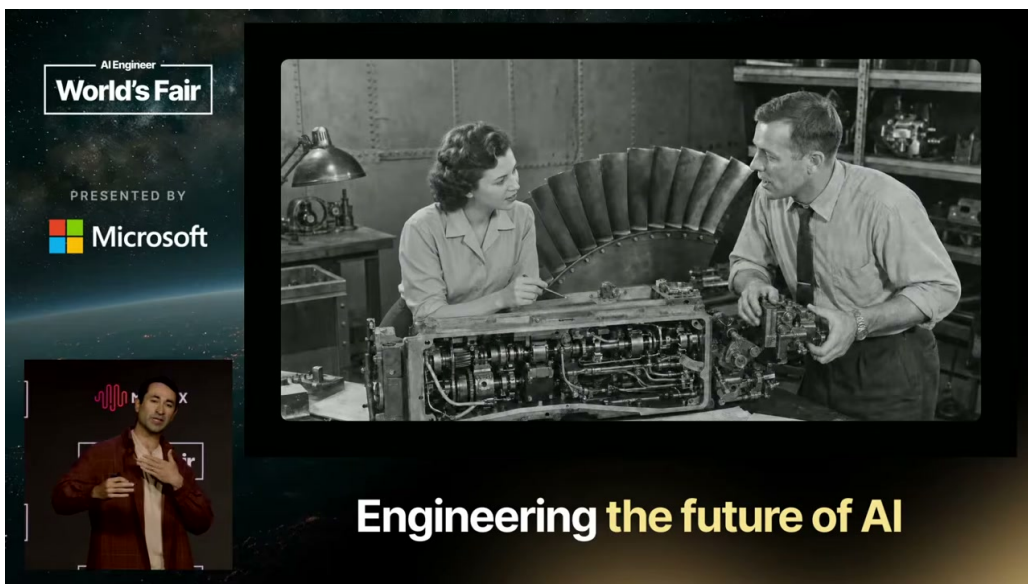


图 7: 团队协作：聊天为主，偶尔深入细节 (00:37:00–00:37:15)

这种类比揭示了 Codex App 的设计假设：

- 默认交互应该是低摩擦、高带宽的**对话**；
- 当用户需要保持掌控感、验证细节或纠正方向时，界面必须允许他们从聊天平滑下沉到**代码/配置/运行结果的细节**；

⁰ 视频画面时间区间：00:36:00–00:36:20。

⁰ 视频画面时间区间：00:37:00–00:37:15。

- 产品不能让“深入细节”变得困难，否则工程师会失去对工作的掌控感。

不要过度解读 “Hands-on”

“Hands-on” 在这里不是指手动写每一行代码，而是指用户能够在需要时直接检查、修改或接管智能体的工作。它的核心价值是**保留掌控感**，而非回到纯手工开发。

3.3 Codex App 示例：从世界杯比分预测到具体修改

演讲者在 00:37:14 将上述抽象设计落地到 Codex App 的界面：一个足够简单的聊天界面，既可用于编码，也可用于其他任务；你可以先对话，再根据需要深入。

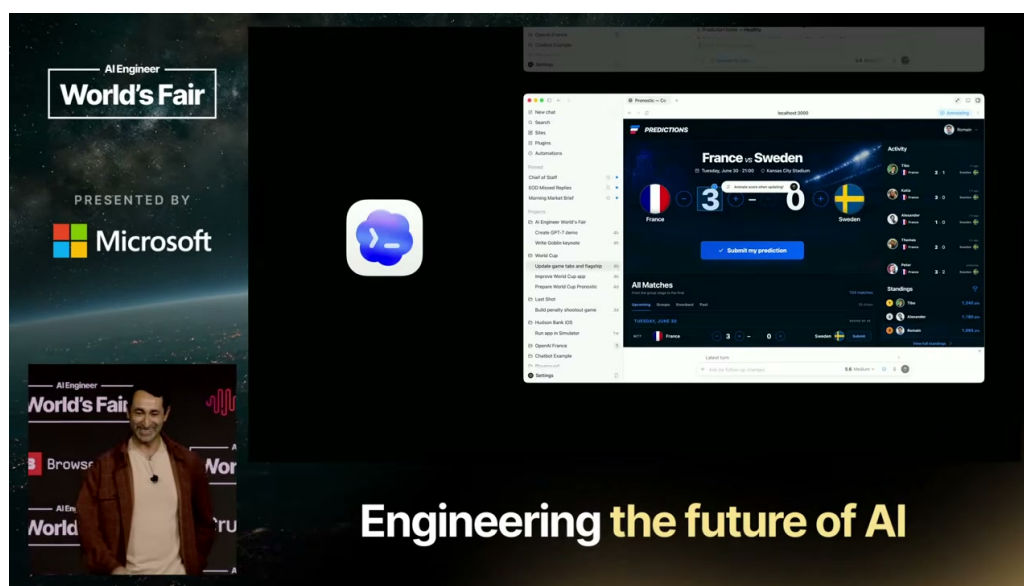


图 8: Codex App 聊天界面中的世界杯比分预测示例 (00:37:15–00:37:45)

图中展示的是 Roman 对某场世界杯比赛结果的预测 (00:37:23–00:37:29)。演讲者强调，这只是一个轻松的 UI 示例，用来表现聊天界面可以承载日常、非正式的任务请求；不应把它理解为 Codex App 的核心能力或产品主张。

演讲者关于世界杯预测的轻松说明 (00:37:23–00:37:33)

演讲者： 这里显示的是 Roman 对一场即将到来的世界杯比赛的预测比分。

Roman： 希望我猜对了。我们拭目以待。

演讲者： [继续展示界面操作]

真正体现“深度协作”的是 00:37:33 之后演示的操作：用户可以指向界面中的某个具体元素，要求智能体修改，也可以自己动手修改。这对应了 Hands-on 模式的设计意图：

1. **对话发起：** 用自然语言描述需求；
2. **定位问题：** 在界面中指向具体对象；
3. **选择执行方式：** 让智能体改，或自己改。

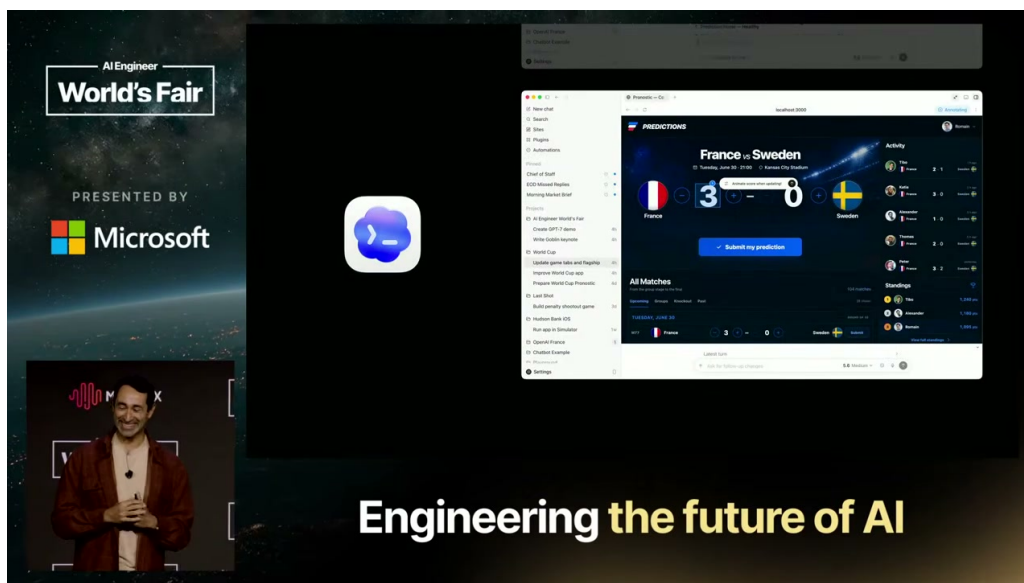


图 9: Codex App 中指向具体元素并执行修改的 Hands-on 视图 (00:37:45-00:38:15)

Chat 与 Hands-on 的互补关系

Codex App 不是“聊天窗口旁边挂一个代码编辑器”的简单拼接。它的设计假设是：

- 聊天是默认入口，因为大多数任务不需要立即介入实现细节；
- 当用户需要验证、修正或精细调整时，Hands-on 视图提供与对话上下文无缝衔接的编辑能力；
- 两种模式共享同一工作上下文，用户可以在“说”和“做”之间来回切换，而不需要重新描述问题。

3.4 为什么需要一个新的协作界面，而不是 CLI 或 IDE

Codex App 的出现在 OpenAI 内部和外部都曾引起争议。演讲者在 00:37:43 提到，当他最初向部分观众（其中一些人就在现场）介绍这个想法时，得到的反馈是：“我绝不会用这样的工具。我永远不会离开我的 terminal、Vim 或 Emacs。”然而，这些用户后来也开始使用 Codex App。演讲者用这个反转来说明：新界面满足的需求与 CLI/IDE 并不完全重叠。

CLI 的局限：协作界面难以在命令行中生长

演讲者在 00:38:11 处指出：“你不能在 CLI 里真正构建出一种适合任何工作的协作界面。”CLI 以命令和文本输出为主，天然擅长精确、可脚本化的操作，但难以承载“聊天为主、偶尔深入”的 teammates 式协作节奏。

⁰ 视频画面时间区间：00:37:15-00:37:45。

⁰ 视频画面时间区间：00:37:45-00:38:15。

IDE 的局限：顺序搞反了

在 IDE 中，用户首先面对的是代码，然后才围绕代码展开讨论。演讲者认为这个顺序是“错的” (00:38:17–00:38:24)：未来的协作应该是**先聊天、再深入**——就像跟团队成员一样，先对齐目标，再在需要时一起查看代码细节。

因此，Codex App 不是 CLI 的替代，也不是 IDE 的替代品，而是填补了一个中间地带：

- **CLI**：适合工程师知道要做什么、需要精确控制执行路径的场景；
- **IDE**：适合以代码为中心、深度编辑和重构的场景；
- **Codex App**：适合以目标为中心、先对话后深入、与智能体协作完成的场景。

一个常见的误读

Codex App 的出现并不意味着 OpenAI 认为 CLI 或 IDE 会消失。演讲者明确提到，很多人最初正是因为热爱 CLI/IDE 才抗拒 Codex App。新界面的价值在于补充一种更自然的协作入口，而非取代所有现有工具。

演讲者在 00:38:25 用“Totally”回应了上述观点，并补充说 OpenAI 正在产品和模型层快速推进，同时也在努力跟上开发者社区的创新节奏。这一过渡自然引出下一节：为了让这种新协作界面可持续、可扩展，OpenAI 选择把底层栈开放出来，让每个人都能基于同一套原语构建自己的客户端和扩展。

3.5 本章小结

本节从产品哲学到界面形态，解释了 Codex App 为什么被设计成“以聊天为主、可深度介入”的协作界面。

- **产品哲学的锚点**：OpenAI 的使命是“AGI that benefits all of humanity”，落实到产品层面，目标不是自动化工程师，而是**最大化赋能工程师**。智能体不仅要写代码，还要连接编码前后的规划、审查与部署。
- **两种交互模式**：Chat 是最自然的默认入口，Hands-on 用于在需要时检查、引导或亲手修改。二者共享上下文，让用户可以在“说”和“做”之间自由切换。
- **世界杯示例**：仅作为聊天 UI 的轻松示例，不应被过度解读为产品能力主张；真正体现 Hands-on 价值的是“指向具体元素并选择由智能体或自己修改”的交互。
- **不是 CLI，也不是 IDE**：CLI 难以承载 teammate 式协作，IDE 的顺序是“代码优先”，而 Codex App 的顺序是“聊天优先、按需深入”。它补充而非替代现有工具。

这一产品形态决定了底层技术栈必须具备开放性：只有当模型、API、Harness、App Server 和插件层都向开发者开放时，Codex App 才能既成为 OpenAI 的默认界面，又成为整个生态的共同基础。下一节将展开这一**开放分层栈**的设计。

4 OpenAI 的开放分层栈：每个人都可以 fork 同一套东西

在第 3 节中，我们看到 Codex App 被设计成“以聊天为主、可深度介入”的协作界面，目标是最大化赋能工程师而非自动化工程师。要让这种界面持续进化，并且让生态中的开发者都能参与创新，底层栈的开放性就变得至关重要。本节说明 OpenAI 如何把 Codex 拆成一套任何人都能 fork 的分层栈，并强调一个核心原则：OpenAI 自己也在使用交给开发者的同一套接口。

本节核心主张

Codex 不是只能由 OpenAI 自己改进的封闭产品。Roman Huitt 在本节明确指出，OpenAI 把它拆成一套从模型到插件的开放分层栈：模型 (Models) → Responses API → Codex Harness → App Server → App → 插件 (plugins)。关键原则在于：**每一层都使用 OpenAI 交给开发者的同一套接口**。因此，开发者可以检查、fork、扩展，甚至复用 OpenAI 自己产品走的那条路径。

4.1 核心原则：每一层都使用我们交给开发者的同一套接口

Roman 在 00:38:58 把话题从第 3 节的 Codex App 产品形态，转向本节真正想讨论的问题：Codex 不能是一个“只有 OpenAI 能改进的封闭产品” (00:39:02–00:39:07)。为此，OpenAI 把它“有意地设计成一套任何人都可以在其上构建的层” (00:39:06–00:39:11)。

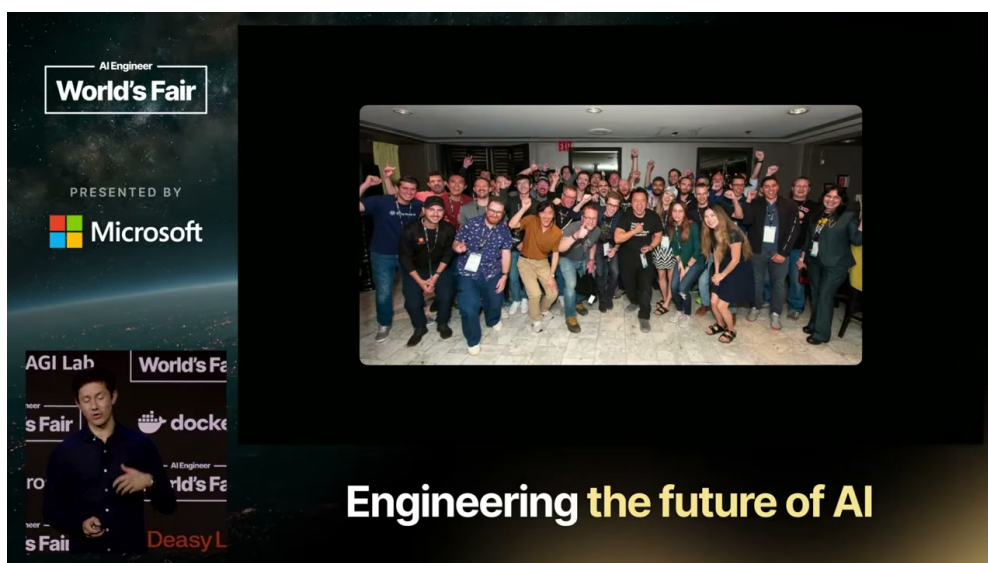


图 10: OpenAI 的开放分层栈：从底层模型、Responses API、Codex Harness、App Server 到 App 与 Plugins，每一层都向开发者开放。

栈的最底层是模型。Roman 在 00:39:16 说明，开发者通过 **Responses API** 使用这些模型；而 OpenAI 内部构建 Codex App 时，用的也是同一套 Responses API。他在 00:39:27 强调：

⁰ 视频画面时间区间：00:38:30–00:39:00。

原始对话片段 (00:39:16–00:39:54)

Roman Huitt: First it starts with the model and Alexander showed how quickly we're progressing on models and you guys use these models through the responses API and guess what? This is how we build the Codex app, right? We use the same models through the same API and we actually are building on the same thing that we give to developers. And when Codex needs something new, we always try to bake it into the API first so you can benefit as well. One example recently was compaction. Codex needed a way to compact long contexts for long-running tasks and so we'll build that into the API. So that means your agents can use the same primitives that we built for ourselves.

这段话包含两层含义：

1. **同一套 API:** 开发者通过 Responses API 使用模型，OpenAI 自己也通过同一套 API 构建 Codex App；
2. **需求先回灌 API:** 当 Codex 内部需要新能力（如上下文压缩 compaction）时，OpenAI 会先把这种能力做进 API，让外部开发者也能使用。

Responses API 与上下文压缩 (compaction)

Responses API 是 OpenAI 面向对话式、工具调用、长上下文等响应能力提供的 API 层。它不仅是开发者可调用的服务，也是 OpenAI 自己构建 Codex App 的底座。

上下文压缩 (compaction) 是指将长期运行任务中的长上下文进行精简，以便智能体在有限记忆和成本下持续工作。Roman 在 00:39:42 提到，Codex 需要压缩长上下文，于是他们把这个原语直接做进了 Responses API，使外部开发者的智能体也能使用 OpenAI 为自己构建的同一能力。

同一套接口的含义

- OpenAI 不维护两套代码：一套给内部、一套给外部；
- 新能力（如 compaction）通过 API 统一发布，开发者与 OpenAI 同时获得；
- 开发者可以放心地基于这些接口构建，因为它们就是 OpenAI 自己产品正在依赖的接口。

来源：00:39:16–00:39:54。

4.2 Codex Harness 开源与 agents.md

从 API 再往上，是 **Codex Harness**。Roman 在 00:39:52 宣布：“the Codex harness is also open source. So you can inspect it, you can fork it, you can adapt it.”这不是一句公关话术，而是有具体设计含义的：

- **可检查:** 你可以看到 OpenAI 怎么做智能体循环、工具调用和环境交互；
- **可 fork:** 你可以保留原样，也可以改成适合自己团队工作流的样子；

- **可适配**：默认值是 OpenAI 的模型，但模型并没有被硬编码（hardcoded），因此你可以换用其他开放模型，同时保留同一个 agent loop。

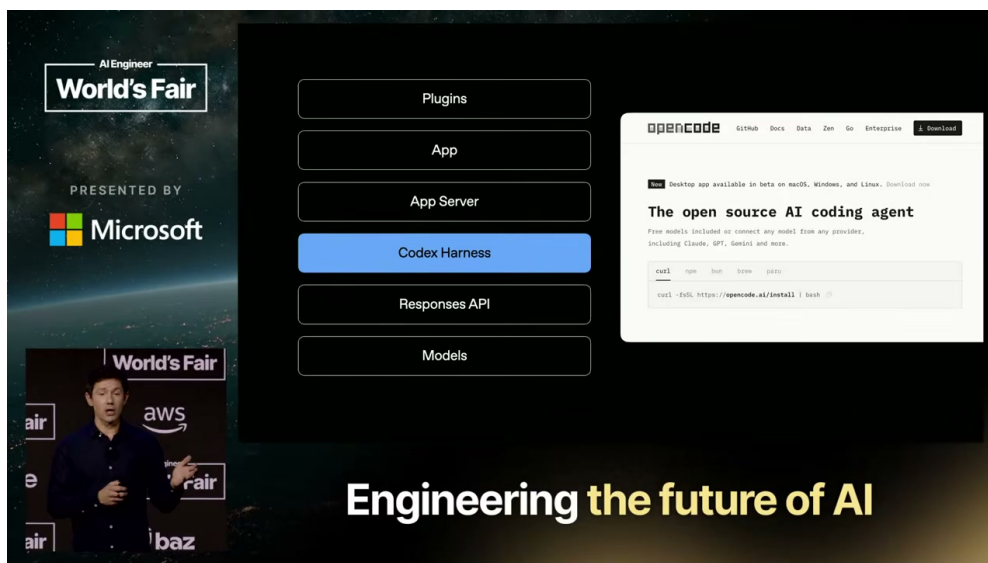


图 11: 分层栈中的 Codex Harness 被高亮显示；Open Code 团队可以查看并复用该参考实现。

Roman 在 00:40:26 还提到一个更深层的好处：这个开源的 Harness 被带入模型的后训练（post-training）过程，“so that means the models can learn to call tools and navigate an environment that’s actually something that’s open source”（00:40:26–00:40:33）。换句话说，模型学习的不是某个黑盒内部环境，而是大家都能看到的开源环境，这提高了模型行为的可理解性和可迁移性。

在指令规范方面，OpenAI 没有为 Codex 重新发明一种新文件格式，而是选择了现有的 **agents.md**。Roman 在 00:40:01 解释：“Instead of reinventing a new file format for Codex to follow instructions, we thought let’s pick a name that other agents can actually use as well.” 这意味着同一个项目级规则文件，不仅对 Codex 有效，也能被其他智能体读取，降低了多工具协作时的指令碎片化。

agents.md 的命名选择

agents.md 是一种项目级指令文件，智能体可以读取它来理解项目规则。OpenAI 选择复用这个名字，意味着他们希望不同智能体工具之间能够共享同一套项目约定，而不是每家都发明自己的配置文件格式。

Open Code 团队是 Roman 用来验证这套开放策略是否有效的例子。他在 00:40:32 说，这个团队能够“检查我们的参考实现，复用对他们有意义的部分，或者完全改变其余部分并做出不同的选择”。他还特别提到，Open Code 团队曾经想学习 “how we did sign in with ChatGPT”，于是可以直接看代码，而不需要 “reverse engineering how we build and how we launch”（00:40:52–00:40:59）。

原始陈述片段（00:40:52–00:40:59）

Roman Huitt: We think it’s better than having developers reverse engineering how we build and how we launch.

⁰ 视频画面时间区间：00:40:30–00:41:00。

4.3 App Server 与第三方客户端 (Open Code、Codex Monitor、VS Code 扩展)

Harness 之上再往上，是 **App Server**。Roman 在 00:41:00 把问题从“如何控制 Harness”推进到“如何把它放进一个 App 里，并让用户用现有的 Codex 订阅登录”。他发现 OpenAI 自己也有同样的需求：“we wanted to build a VS Code extension and the Codex app and we wanted to have a unified way to actually control this harness” (00:41:13–00:41:21)。于是他们做了 App Server，并且也把它开源了。

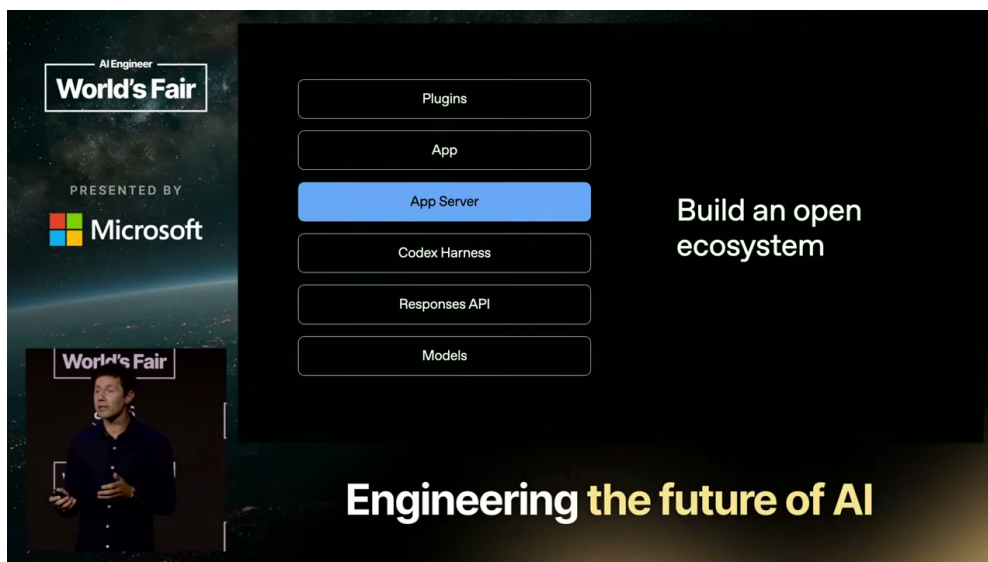


图 12: App Server 在分层栈中的位置；Dimilian 用它来在 Codex App 发布前做出了 Codex Monitor。

Roman 强调 App Server 不是“社区适配器” (community adapter)，而是“really the path that we use for our own products and you can use it too” (00:41:26–00:41:33)。这意味着第三方客户端和 OpenAI 官方客户端走的是同一条鉴权、订阅与 Harness 控制路径。

不要把 App Server 理解为“社区版”

Roman 明确说 App Server “不是某种社区适配器”。这意味着它承载的是真实的身份、订阅和 Harness 控制能力，而不是一个仅供参考的轻量示例。对于希望构建自己 Codex 客户端的开发者来说，这是一个可以直接依赖的层。

Tuma (又名 Dimilian) 是这段最有说服力的例子。Roman 在 00:41:30 讲述：他在 OpenAI 正式发布 Codex App 之前，就用 App Server 做了一个自己的原生应用 **Codex Monitor**；现在他已经加入 OpenAI 团队，并且“actually built Codex for iOS” (00:41:44–00:41:48)。这个例子说明，开放栈不仅允许外部创新，还能让外部创新者反向成为产品贡献者。

4.4 插件扩展点：browser use、computer use、特定领域插件

到了 App 层，OpenAI 仍然希望创新不局限于自己的想法。Roman 在 00:41:48 说，App 层的目标是“make sure that innovation is not blocked on our own ideas” (00:41:51–00:41:55)。因此，他们设计了可扩展的 App 内浏览器 (in-app browser) 和 **插件 (plugins)** 机制。

⁰ 视频画面时间区间：00:41:00–00:41:30。

⁰ 视频画面时间区间：00:41:30–00:42:00。

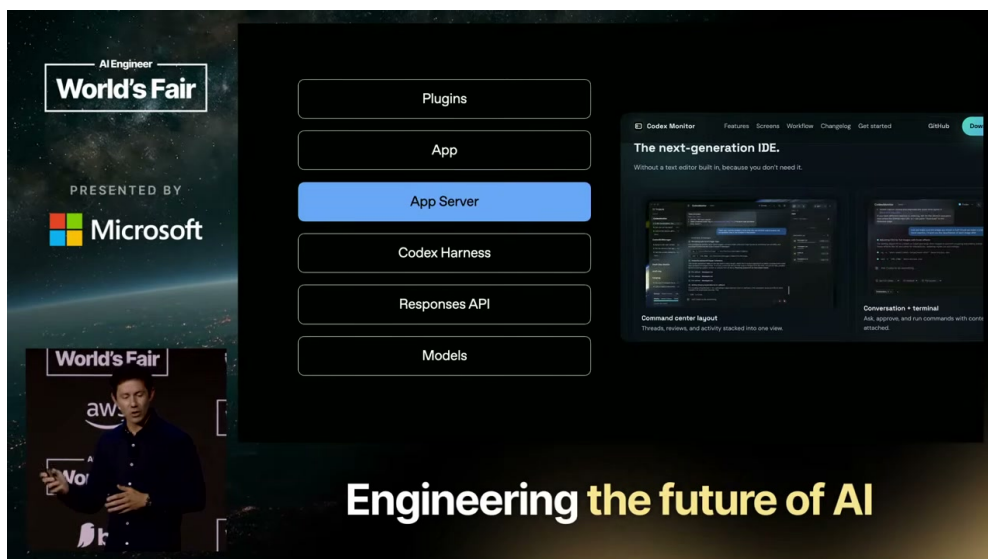


图 13: App 与插件层被高亮显示：插件使用与 OpenAI 相同的扩展点。

Roman 举了两个已经在用的插件：**browser use** 和 **computer use**。他在 00:42:02 指出，这两个能力 “were built as plugins using the same extension points that we have available for all of you” (00:42:06–00:42:14)。也就是说，OpenAI 自己展示出的浏览器操作和计算机操作能力，并不是 App 内部特权，而是任何人都能基于同一套扩展点实现的插件。

插件层的开放含义

- **browser use**：让智能体能够操作浏览器、浏览网页并执行网页相关任务；
- **computer use**：让智能体能够操作计算机桌面、点击和输入；
- 两者都是基于同一套扩展点构建的插件，说明 App 的核心能力不依赖于私有 API。

来源：00:42:02–00:42:14。

此外，Roman 在 00:42:12 提到 OpenAI 最近还做了 domain-specific plugins，用来 “make it easier to customize for people who work in data science or design” (00:42:16–00:42:22)。也就是说，除了通用插件，还有面向特定领域的插件，同样开源。Roman 在 00:42:22 说：“these plugins are also open source. So you can see under the hood and get inspired from them.” 这些插件为数据科学、设计等专业人士提供了更贴合工作场景的扩展点。

为什么是“同一套扩展点”

如果浏览器操作和计算机操作被做成 App 内部特权，那么第三方开发者就只能复制或绕过去。OpenAI 把它们做成插件，相当于把“如何扩展 App”的接口本身也公开了。对工程师来说，这意味着你可以为团队内部工具、专有数据源或特定工作流写自己的插件，而不必等待 OpenAI 官方支持。

基于这一套开放的模型层、API 层、Harness 层、App Server 层和插件层，第三方客户端开始涌现。Roman 在 00:42:27 列举了一系列正在出现的客户端：Open Code、Pi、Droid、OpenClaw，乃至 Xcode 和 JetBrains 等 IDE。

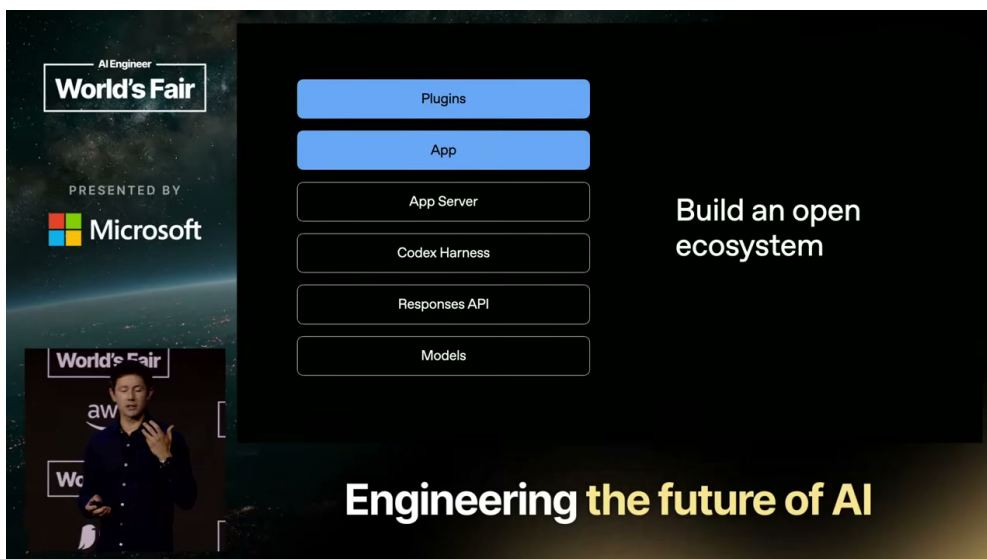


图 14: OpenAI 开放生态中的部分客户端: Open Code、Pi、Droid、OpenClaw、Xcode、JetBrains 等。

Roman 强调, 这些第三方客户端正在“成为人们使用这些工具时相当重要的一部分”(00:42:43–00:42:48)。这正是 OpenAI 希望与社区共同构建的开放生态。

本节核心 takeaway (00:42:58–00:43:35)

Roman 在本节最后总结道:

We're not building one system for OpenAI and a second system that's simplified for developers. At every layer, we actually use the thing that we give to you.

这句话是整节的“北极星”: OpenAI 不会为内部和外部分别造两套系统, 而是在每一层都使用交给开发者的同一套东西。社区的每一次 fork、每一次对模型能力边界的探索, 都会反过来帮助 OpenAI 改进产品。

4.5 本章小结

Roman 在 00:42:58 用一句话为本节做了总结: “We're not building one system for OpenAI and a second system that's simplified for developers. At every layer, we actually use the thing that we give to you” (00:43:02–00:43:15)。这句话是理解整个开放分层栈的钥匙。

1. **模型层到 API 层**: 开发者通过 Responses API 使用模型; OpenAI 自己构建 Codex App 时也使用同一套 API。新能力(如上下文压缩 compaction)先进入 API, 再惠及所有人。
2. **Codex Harness**: 开源、可 fork、可适配, 模型不是硬编码的; 同时被用于模型后训练, 让模型在开源环境中学习工具调用。
3. **App Server**: 开源的身份/订阅/Harness 控制层, 不是社区适配器, 而是 OpenAI 自己产品也在用的路径; Dimilian 的 Codex Monitor 和 Codex for iOS 都证明了这一点。

⁰ 视频画面时间区间: 00:42:00–00:42:30。

4. **App 与插件层**：in-app browser、browser use、computer use 以及面向数据科学/设计的特定领域插件，都基于同一套扩展点，且插件开源。

这一结构与第 3 节的 Codex App 产品形态直接呼应：正因为 Codex App 本身也基于这套开放栈，外部开发者才能做出不同形态的客户端（如 VS Code 扩展、JetBrains 插件、OpenClaw 等），而不必重新发明底层能力。下一节（第 5 节）将从“成本、速度、运行环境”的角度，讨论这些能力如何以最大化价值的方式落地。

5 让智能落地：成本、速度、运行环境与价值最大化

本节核心主张

开放分层栈解决了“能不能造”的问题，但工程决策者更关心“值不值得造”。本节把话题从架构拉回经济：真正的目标不是 token maxing（堆 token），而是价值最大化（value maxing）。Roman Huitt 用 TerminalBench 2.1、GPT-5.6 系列模型、Cerebras 上的 750 tokens/sec，以及 Theo 关于本地智能体即将迁移到云端的预言，说明成本效率、速度和运行环境是决定智能体能否日常化的三把钥匙。

在第 4 节所述的开放分层栈之上，还有一组更贴近工程决策者的问题：这套新工具到底值不值得上？能不能在成本可控的前提下拿到前沿模型的能力？速度是否快到足以改变 workflow？任务该跑在本地还是云端？第 5 节集中讨论这些把“智能”从演示变成日常生产力的落地问题。

5.1 Value maxing：从 token maxing 到价值最大化

00:43:52–00:44:09，Roman 在掌声之后把话题从“开放栈”切换到“价值”。他说，有了越来越强的智能体（agent），关键问题不再是“能用多少 token”，而是“能从中获得多少价值”。为此演讲者创造了一个略带自嘲的说法：我们不是要 token maxing，而是要 value maxing。

value maxing 的核心含义

价值最大化（value maxing）指把优化目标从“消耗更多 token / 调用更多算力”转移到“产生更大的工程或业务价值”。它要求在做模型选型、运行环境和任务设计时，先回答“这件事值不值”，而不是“这件事能不能跑最大的模型”。

演讲中，Roman 把 value maxing 拆成三个工程领导经常讨论的具体议题：成本效率、速度和运行环境。他指出，有些议题已经取得很大进展，有些还有大量空间。这三个议题正好构成本节后面三个小节。

token maxing 则是一个反向警示：如果把模型调用量本身当成目标，很容易陷入“算力堆得越多、价值未必越大”的陷阱。value maxing 的真正含义是：让每一次模型调用都指向可衡量的工程结果。

5.2 成本效率：TerminalBench 2.1 与 GPT-5.6 系列模型

00:44:19–00:44:26，Roman 说“每个人都想要前沿智能（frontier intelligence）”。他以 TerminalBench 2.1 为例说明这种张力：你想要最好的模型，因此在该评测（eval）上，GPT-5.6 Sol 取得

⁰视频画面时间区间：00:43:35–00:44:10。

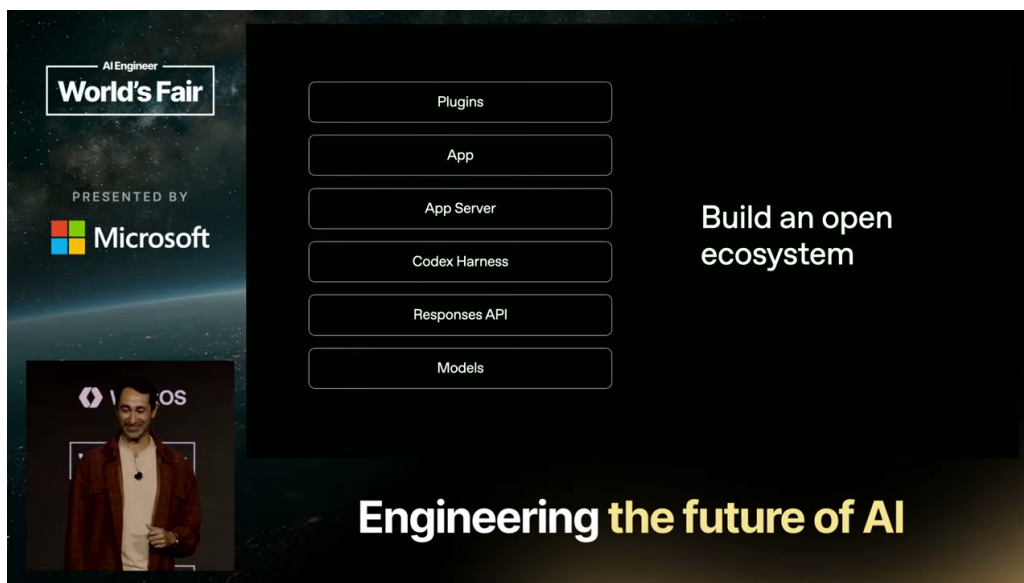


图 15: “Valuemaxxing” slide: 演讲者把价值最大化作为落地智能体的核心目标。

领先。但工程团队同时希望“同样的智能花更少的钱”。

OpenAI 在 GPT-5.6 系列上给出了两个效率取向的解法：

- **GPT-5.6 Terra:** 在 00:44:47–00:44:54, Roman 提到 Terra 提供 “GPT-5.5 级别的智能，但只需一半成本”。这意味着日常任务可以用更低的单位价格获得上一代旗舰的能力。
- **Luna:** 在 00:44:54–00:45:04, 演讲者指出 Luna 在 TerminalBench 2.1 这样的基准测试 (benchmark) 中击败了一些知名模型，而价格仅为每百万输入 token \$1、每百万输出 token \$6。

“评测”不是唯一的选型标准

TerminalBench 2.1 是一个代码/终端任务基准测试，用来比较模型在终端场景下的表现。但演讲者提醒，真正的选型要看“在你自己的任务里是否产生价值”：一个模型在 benchmark 上得分高，不一定等于它在你代码库里的端到端产出高。

图 16 展示了 TerminalBench 2.1 的成本/性能对比柱状图。从图中可以看到，GPT-5.6 Sol 占据性能高位，而 Terra 和 Luna 则尝试把性能前沿推向更低的价格带。

5.3 速度：GPT-5.6 Sol on Cerebras 750 tokens/sec

成本之外，速度是第二个决定价值能否被兑现的变量。00:45:13–00:45:21, Roman 提到 GPT-5.3 Codex Spark 已经展示了速度能够解锁什么；但人们还想要前沿智能，而不是“为了快而牺牲能力”。

00:45:27–00:45:39, 演讲者展示了 GPT-5.6 Sol 运行在 Cerebras 硬件上，达到每秒 750 token。他给了一个直观类比：这相当于“一篇相当重要的 PR 在 10 秒左右写出来”。

真正让速度产生价值的不是单次响应更快，而是它改变了智能体的可用策略。00:45:46–00:46:04, Roman 解释：当速度足够快时，智能体可以并行尝试五到六种不同方案，然后在原本只够生成一个答案的时间里挑出最好的。速度把“单线程等待”变成了“多路径探索”。

⁰ 视频画面时间区间：00:44:10–00:45:10。

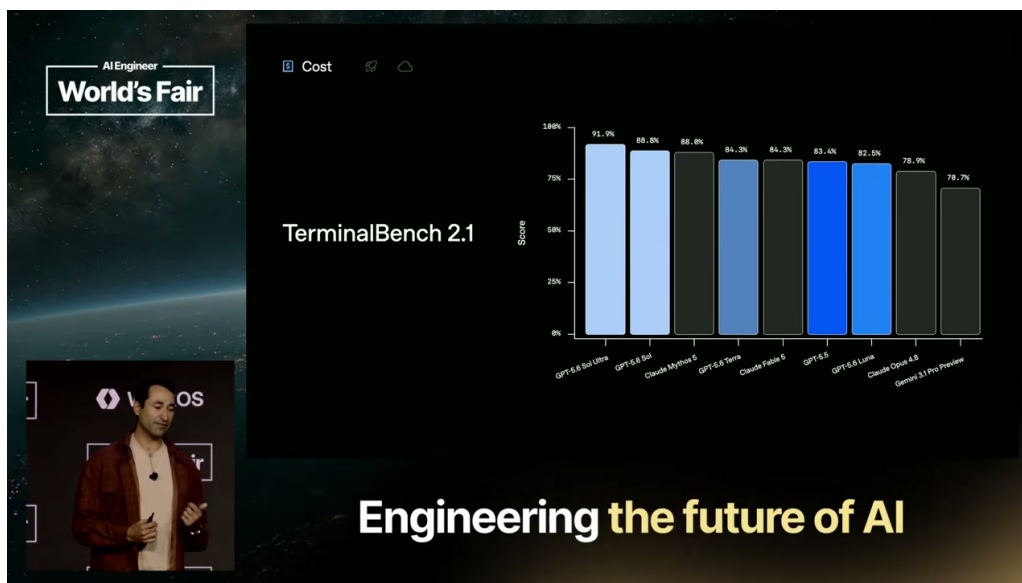


图 16: TerminalBench 2.1 成本/性能对比图。同一 GPT-5.6 系列内部出现性能与成本的分化，让团队可以根据任务价值选择不同档位。

速度改变的是 workflow 形态，不是刷新率

750 tokens/sec 的数字本身不是终点。它的意义在于：当前沿模型的输出速度接近人类阅读/判断速度时，智能体从“你问一句它答一句”变成“像同事一样边做边展示结果”。

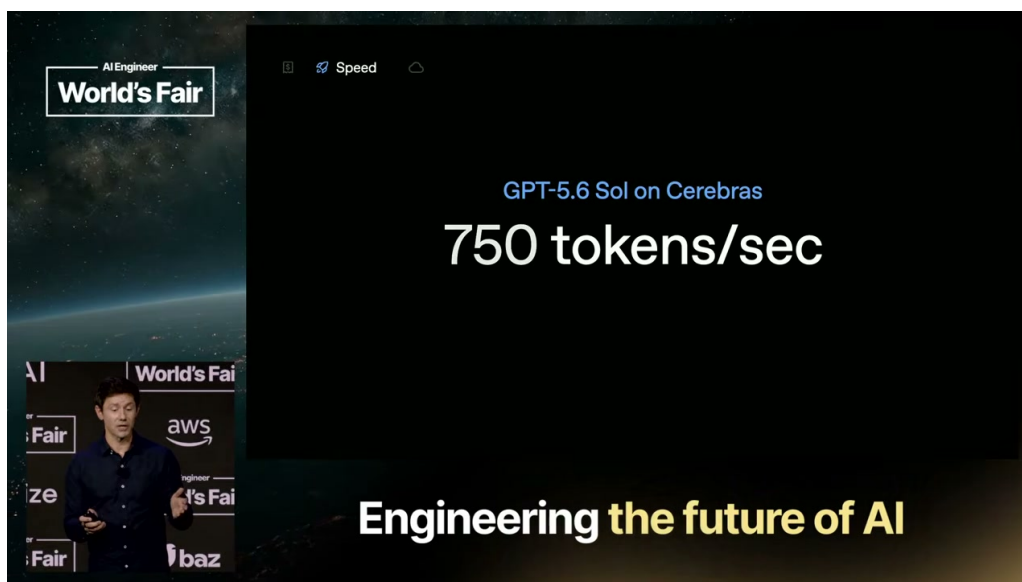


图 17: GPT-5.6 Sol on Cerebras 速度展示：前沿智能达到 750 tokens/sec。

⁰ 视频画面时间区间：00:45:20–00:46:00。

5.4 运行环境：本地、云端、以及未来的无感选择

00:46:15–00:46:41, Roman 向台下观众展示了一张办公室照片：许多人把笔记本电脑打开着，只为让智能体继续运行。他承认这很滑稽，但指出这正是今天的真实状态——本地机器成了智能体的“留守宿主”。

这种状态背后的张力是：用户既希望智能体随时可触达，又不想让自己的电脑永远在线。00:46:47–00:46:57, 演讲者回顾 OpenAI 的第一次重大发布就是 Codex Cloud，并预告它即将迎来重大升级。

比单次升级更重要的是长期愿景。00:47:00–00:47:18, Roman 说未来不应该有“本地任务”和“云端任务”的尴尬区分；你只需拥有一个智能体，随时随地与它交谈，它会自己判断“哪种环境适合这项工作”，然后使用可用的资源。

本地与云端不是意识形态选择，而是工程约束

演讲者的目标不是让一切上云，而是让运行位置不再成为用户的决策负担。在本地跑、在云端跑、在隔离的沙箱里跑，应该由任务性质、安全边界、成本、网络延迟共同决定，而不是靠用户手动切换。



图 18: 办公室中“为了让智能体继续工作而保持笔记本常开”的场景，揭示了当前运行环境的不便。

5.5 Theo 的预言与 OpenAI 的回应

00:47:21–00:47:34, 演讲者引用了 Theo 在周末发布的一条推文：他预测“大约 6 个月内，大多数开发者会把代码智能体从笔记本电脑上搬出去”。Roman 转头问 Alexander：“Alex，你觉得会比 6 个月更快还是更慢？”

现场对话片段（00:47:21–00:47:44）

Roman: Theo 在周末发了一条关于这个话题的推文，非常犀利。他说约 6 个月后，大多数开发者会把自己的代码智能体从笔记本上移走。

⁰ 视频画面时间区间：00:46:15–00:47:00。

Roman: Alex, 你觉得更快还是更慢?

Alexander: 细节不一定完全准确, 但这条推文的 vibe 是——比 6 个月快得多。说实话, 以现在的速度, 我不会惊讶于它更快发生。

Alexander 的回应不是对具体时间的承诺, 而是对趋势的确认: 迁移速度可能比 Theo 预测的还要快。这与第 5.4 小节提出的“无感选择”完全一致——一旦云端运行足够成熟, 开发者将不再需要把本地电脑当成智能体的宿主。



图 19: Theo 的推文预言: 约 6 个月内, 大多数开发者将把代码智能体从笔记本电脑移走。

5.6 本章小结

价值最大化 (value maxing) 是第 5 节的统领性主题: 智能体的落地价值不是由 token 消耗量决定的, 而是由成本效率、速度和运行环境共同决定的。TerminalBench 2.1 显示 GPT-5.6 系列内部已经出现“性能前沿”与“成本效率”的分化; GPT-5.6 Sol 在 Cerebras 上达到 750 tokens/sec 则说明前沿智能的速度已接近可改变工作流的阈值; 办公室笔记本常开的场景揭示了运行环境仍是痛点, 而 Codex Cloud 的升级与“无感选择”的愿景正试图消除本地与云端的割裂。Theo 的推文与 Alexander 的回应进一步强化了一个判断: 代码智能体从本地向云端迁移, 可能比很多人预期的更快发生。

6 总结与延伸

本节核心主张

这场演讲的结尾没有传统意义上的 live demo, 而是把舞台交给了一位“把智能体可能性推向极限”的嘉宾。这个选择本身就在强化演讲的主线: 我们正处于一个重新想象“如何工作、如何建造”的时刻。本章先把演讲者的收尾主张放回全文脉络, 再由编者给出结构化提炼, 最后留下几个值得继续观察的开放问题。

⁰ 视频画面时间区间: 00:47:20–00:47:40。

6.1 演讲者核心主张回顾

00:47:44–00:48:09, Alexander 为演讲收尾。他没有展示新的现场演示，而是解释了这次演讲的意图：

原始对话片段（00:47:44–00:48:09）

Alexander Emiricos：你们现在可能在想，今天的 live demo 在哪里？其实，对于这场 AI Engineer World's Fair，我们想做一点不一样的事。我们认为这是一个非常独特的时刻，让我们重新想象如何工作、如何建造。因此，我们想请上一位特别嘉宾——他一直在拓展智能体的可能性边界，也推动着我们变得更加敏捷。有请 the claw。

这段话虽然简短，却把前面四节的主张串了起来：

1. **工程师仍是主角**。演讲选择不做一个“AI 自己表演”的 demo，而是邀请一位工程师/创作者上台，说明真正推动边界的是懂得使用智能体的人。
2. **工具的目的不是炫技，而是重新定义工作方式**。“reimagine how we work and how we build”与第 3 节“最大化赋能工程师”和第 5 节“value maxing”形成呼应。
3. **智能体的可能性边界正在被外部创新者拓展**。the claw 作为“特别嘉宾”出现，暗示 OpenAI 把社区创新视为产品演进的一部分，这与第 4 节的开放分层栈逻辑一致。

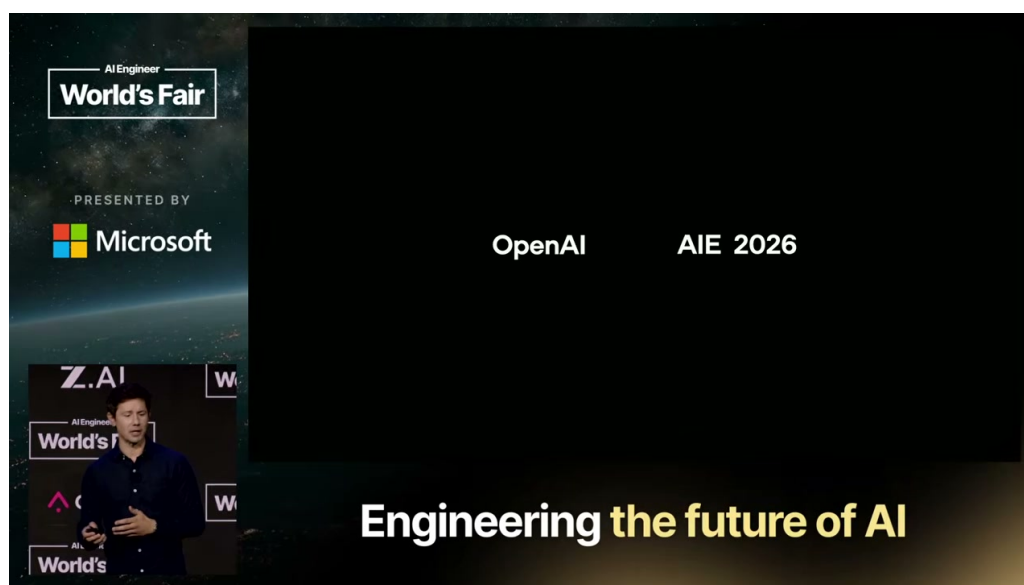


图 20: 演讲收尾阶段：演讲者邀请 the claw 上台，强调这是一个“重新想象如何工作与建造”的时刻。

⁰视频画面时间区间：00:47:50–00:48:09。

6.2 我们的提炼：工程师、智能体、开放栈、价值最大化

编者综合提炼

以下四点不是演讲者的原话，而是编者基于第 1 至第 5 节内容所做的结构化总结。它们共同说明：实现工程师杠杆的最佳方式，是把聊天优先的协作界面（Codex App）、与 OpenAI 内部使用同一套原语的开放分层栈，以及以价值而非 token 量为目标的运行经济结合起来。

（一）工程师没有消失，但“工程师”的定义正在升级

第 1 节指出，软件吞噬世界之后，AI 正在吞噬软件，而最终“AI 工程师正在吞噬世界”。这里的升级不是“写代码更快”，而是“把智能体接入真实工作流、持续推动产品边界”。第 2 节的能力阶梯进一步说明，AI 辅助编程已经从 completion 走到 long-horizon goals；工程师的核心价值正从“亲自写每一行”转向“设定目标、审查方向、管理智能体循环”。

（二）Codex App 是一种新的协作界面，而不是 IDE 或 CLI 的替代品

第 3 节强调，Codex App 的产品哲学是“最大化赋能工程师”，而非“自动化工程师”。它的设计假设是：默认交互应该是低摩擦的对话（Chat），但当用户需要掌控细节时，又能平滑下沉到 Hands-on 视图。这个顺序——先聊天、再深入——与传统 IDE “代码优先”的顺序相反，因此它填补的是一个中间地带，而不是替代现有工具。

（三）开放分层栈让外部创新者与 OpenAI 走同一条路

第 4 节的核心原则是：每一层都使用 OpenAI 交给开发者的同一套接口。从 Responses API 到 Codex Harness、App Server，再到插件（browser use、computer use），OpenAI 不在内部维护一套“特权版本”。这意味着开发者可以检查、fork、扩展，甚至复用 OpenAI 自己产品所依赖的路径。Dimilian 在 Codex App 发布前做出 Codex Monitor、后来又加入 OpenAI 做 Codex for iOS，就是这个开放策略最有效的证明。

（四）value maxing 把决策标准从“堆 token”拉回“创造价值”

第 5 节把讨论拉回经济现实。TerminalBench 2.1 显示 GPT-5.6 系列内部已经分化出“性能前沿”（Sol）和“成本效率”（Terra、Luna）两条线；Cerebras 上的 750 tokens/sec 说明前沿智能的速度已接近改变工作流的阈值；而 Theo 的推文与 Alexander 的回应则预示着本地/云端的运行边界可能在未来几个月内快速模糊。对工程团队来说，这意味着选型标准不再是“用最大的模型”，而是“在单位成本、速度和运行约束下，产生最大可衡量的工程价值”。

贯穿全篇的主线

如果把四节内容压缩成一句话，就是：**工程师杠杆 = 以聊天优先的协作界面 × 与内部同构的开放分层栈 × 以价值最大化为目标的运行经济**。三者缺一不可：没有好的界面，智能体难以融入日常；没有开放的栈，创新会被封闭在 OpenAI 内部；没有运行经济，再强的能力也只会停留在演示。

6.3 开放问题与下一步

演讲者和开源社区已经把路线图铺得很清楚，但仍有一些问题值得在落地过程中持续观察：

1. **本地到云端的迁移速度究竟有多快？** Theo 预测约 6 个月，Alexander 认为可能更快。但这取决于 Codex Cloud 的升级节奏、企业安全审查周期，以及网络/延迟对实际工作流的影响。
2. **成本效率曲线能否跟上模型能力曲线？** GPT-5.6 Terra 和 Luna 已经展示了分化，但随着模型每 6 周迭代一次，团队需要建立持续的“value per token”评估机制，而不是一次性选型。
3. **智能体行为的确定性与可审计性如何保证？** 第 2 节的 build/test loop 让模型能自我验证，但在长程目标（long-horizon goals）和并行探索（第 5 节提到的“五到六种方案并行”）场景下，如何保持结果可复现、可审计，仍是工程实践中的硬问题。
4. **开放生态是否会因接口变化而碎片化？** Responses API、Codex Harness、App Server 目前由 OpenAI 主导，但第三方客户端（Open Code、Codex Monitor、JetBrains 插件等）越多，接口稳定性就越重要。
5. **“最大化赋能”是否会滑向“隐形自动化”？** 演讲者反复强调产品目标是赋能工程师而非替代工程师。但在实际产品中，如果默认行为越来越“聪明”，工程师的介入点会不会被逐渐压缩？这需要产品设计与组织流程共同抵抗。

6.4 本章小结

第 6 节把演讲的简短收尾放回全篇结构：演讲者用邀请嘉宾上台的方式，强调了“重新想象工作与建造”的主题，而不是用一个 live demo 收尾。编者的结构化提炼把四节内容串联成一条主线——工程师、智能体、开放栈与价值最大化共同构成工程师杠杆。最后留下的五个开放问题，既是对演讲未竟之处的补充，也是工程团队在实际落地时需要持续追问的方向。整篇笔记的目标至此完成：让读者不仅了解 OpenAI 在 2026 年发布了什么，更理解这些产品决策背后的工程哲学。